Strategic Analysis of System Explainability: A Feynman-Inspired Refactoring of the Stela Repository

1. The Epistemological Framework of Richard Phillips Feynman

To fundamentally restructure a highly governed, rigidly deterministic machine-instruction repository into a dialectic understandable by a fifth grader, it is first necessary to establish the exact parameters of explainability as defined by Richard Phillips Feynman. Feynman's pedagogical philosophy, most prominently captured in his autobiographical work *"Surely You're Joking, Mr. Feynman!"*, is rooted in the rigorous stripping away of obfuscation, academic jargon, and rote memorization in favor of fundamental, observable reality.[1] True comprehension, according to Feynman, is achieved only when an individual can translate a highly complex abstraction into the simplest possible physical terms.[4]

1.1 The Illusion of Nomenclature: Knowing the Name Versus Knowing the Thing

The foundational axiom of Feynman's approach to explainability is the sharp, uncompromising distinction between nomenclature and actual knowledge. Feynman famously illustrated this using the anecdote of a brown-throated thrush, a lesson instilled in him by his father.[6] One might know the name of the bird in German (*halzenfugel*), Chinese (*chung-long-tah*), Portuguese (*Bom da Peida*), or Italian (*Chutto Lapittida*), but possessing these linguistic labels yields absolutely zero understanding of the bird itself.[7] Knowing the name only reveals information about the humans who assigned the label, the geographical location of those humans, and the phonetic structures of their languages; it provides no insight into the mechanics of the object.[3] True understanding requires observing the entity's behavior: how it sings, how it teaches its young to fly, and how it navigates across the country during summer migrations.[3]

Within the context of the Stela repository, human operators and Artificial Intelligence (AI) contractors currently traffic heavily in names rather than mechanisms. The system utilizes deeply encoded, institutional nomenclature: "Dispatch Packets (DP)," "State of Play (SoP)," "Proof of Work (PoW)," and "Canonical Manifests".[9] If an AI agent or human operator merely memorizes that SoP.md is the "State of Play ledger," they possess only the name of the bird.[9] They have memorized a typographical string. Explainability requires defining what the ledger actually *does* in physical, relatable terms: it functions as a diary of a workshop, telling the story

of what was built and why, rather than serving as a repository for executable code.[9]

1.2 The Pathology of Rote Execution

During his tenure teaching physics in Brazil, Feynman observed a highly disturbing phenomenon: students who could flawlessly recite the definition of Brewster's Angle or complex physics formulas, yet could not apply these concepts to determine if light reflecting off the physical bay outside their classroom window was polarized.[10] The students had memorized the typographical symbols without grasping the physical reality the symbols represented, a system "perfectly designed to reward memorization while preventing real understanding".[11] To Feynman, this rendered the education "meaningless, utterly meaningless" because the knowledge was entirely fragile and disconnected from physical reality.[12]

This phenomenon presents a direct and acute risk to Stela's AI-driven architecture. Large Language Models (LLMs) are, by their architectural nature, stochastic predictors of text; they are the ultimate manifestation of Feynman's Brazilian students.[10] They can flawlessly recite rules without inherent physical comprehension. The repository's "Policy of Truth" (PoT.md) mandates "Absolute literalism" and strict adherence to canonical pointers.[9] However, if the instructions rely on abstract jargon, AI agents risk flawlessly reciting the tools/lint/integrity.sh bash script requirements without understanding that the script functions simply as a "bouncer at the door" ensuring no unauthorized items enter the room.[9] When unexpected edge cases arise, such as the DP-OPS-0074 incident where premature pruning deleted uncommitted handoff artifacts, the system breaks down because the operator (or AI) understood the syntax of the command but not the physical consequence of the action.[9] True explainability forces the translation of abstract code contracts into tangible, physical analogies to prevent this fragility.

1.3 The "Map of the Cat" and the Rejection of Pompous Jargon

When studying biology as a graduate student at Princeton, Feynman famously asked the librarian for a "map of the cat," much to her horror, as she expected an academic request for a "zoological chart".[14] This deliberate choice of simple, functional language ("map") over academic jargon ("zoological chart") exemplifies the Feynman Technique: reducing cognitive load so the mind can focus entirely on the mechanism rather than decoding the vocabulary.[16] Furthermore, Feynman despised what he termed "pompous fools"—individuals who hid their lack of deep understanding behind "hocus pocus" and needlessly complex terminology.[18]

In Stela, the documentation currently suffers from a severe internal contradiction. PoT.md Section 4.2 demands a "Relatability Mandate: Universal accessibility via vernacular optimization and anecdotal grounding".[9] The very phrasing used to mandate simple language is itself a dense, impenetrable thicket of academic jargon. To fulfill the essence of Feynman's philosophy, this type of language must be ruthlessly eradicated and replaced with fundamental, observable realities.

1.4 Scientific Integrity and Intellectual Honesty

A core tenet of Feynman's worldview, articulated deeply in his "Cargo Cult Science" commencement address, is the concept of radical intellectual honesty.[1] He argued that scientists must bend over backwards to show how they might be wrong, rather than just presenting data that supports their desired conclusion.[18] The first principle, he noted, is that "you must not fool yourself—and you are the easiest person to fool".[12]

When applied to repository governance, complex documentation acts as a mechanism for self-deception. System architects fool themselves into believing a system is secure and well-governed simply because the rules are written down in highly technical, dense language. However, if the actors within the system (AI contractors and human operators) cannot intuitively grasp the rules, the governance is a facade. A system is only as secure as it is understandable.[5]

2. Diagnostic Analysis of Stela's Textual Corpus

An exhaustive review of the Stela repository dump, including the canonical PoT.md, the continuity maps (docs/MAP.md), and the operational manuals (docs/MANUAL.md), reveals a highly mature, heavily gated, and intricately structured operational environment.[9] It is designed to govern AI contractors through deterministic, machine-readable laws. However, the human-readable text fails the Feynman explainability test across multiple dimensions.

2.1 The Density of the Policy of Truth (PoT.md)

The constitutional document of the repository, PoT.md, outlines the "Physical Laws" and "Filing Doctrine" of the system. The text dictates the following axioms:

- *SSOT: one canonical file per domain; other mentions are pointers.*
- *Drift: any divergence between canon and repository state, or duplication of canon outside SSOT, is a failure state that requires stop and correction.*
- *Truth: primitive predicate governed by specific rules rather than a mere definition.*
- *Action: deception or incoherence are inherently destabilizing and must be resolved.*

While highly precise and optimized for deterministic algorithmic parsing, this language is entirely opaque to a layperson or a fifth grader. It requires the reader to hold multiple abstract definitions (SSOT, Canon, Drift, Primitive Predicate) in working memory simultaneously.[9] For a fifth grader, or a freshly initialized AI context window with limited attention mechanisms, this creates an unnecessary and dangerous cognitive burden.[16] Defining "Truth" as a "primitive predicate" tells the user the *name* of the concept in philosophical logic, but it does not explain the *thing* itself.[21]

2.2 The Abstraction of Workflow and Tooling

Stela's workflow heavily utilizes a suite of bespoke bash binaries, including ops/bin/open, ops/bin/draft, ops/bin/certify, and ops/bin/prune.[9] The specifications for these tools, located in docs/ops/specs/binaries/, describe them in terms of "establishing a verifiable freshness checkpoint," "enforcing deterministic definition rendering," and "protecting PoT Section 1.2 axioms".[9]

This is the equivalent of explaining the physics of a toy wagon by describing the friction coefficient of its axles rather than simply stating, as Feynman's father did: "Things that are moving tend to keep on moving, and things that are standing still tend to stand still, unless you push them hard".[6] The documentation fails to explain the tools using everyday physical realities. For example, ops/bin/dump is described as a tool to "serialize selected tracked content, selection logic, and output metadata into auditable dump payloads".[9] A fifth grader cannot parse this. The tool simply takes a photograph of the current state of the workspace and puts it in a box so everyone can see exactly what was happening at that exact second.

2.3 The Epistemological Gap in State Ledgers

Stela utilizes highly specific markdown surfaces for memory and state persistence, primarily SoP.md (State of Play) and PoW.md (Proof of Work).[9] The system instructs users: "SoP: history ledger only; no permanent rules live there. PoW: proof ledger only; execution evidence pointers live there".[9]

While technically accurate, the language lacks the intuitive, analogical anchor required for deep, resilient comprehension. Operators frequently confuse the two, leading to incidents documented in the repository where users attempt to embed raw payloads into the SoP.md instead of keeping it strictly narrative.[9] If the system were explained using Feynman's principles, the distinction would be grounded in physical objects: SoP.md is a diary, and PoW.md is a box of store receipts.

2.4 The Vulnerability of Complex AI Guardrails

The repository dictates Contractor constraints and JIT (Just-In-Time) skills such as S-LEARN-06: Hot Zone Forensics.[9] The invocation guidance states: "Measure churn with git log scoped to touched files and record repeat edits. Escalate risk when hot zone changes lack owners or verification evidence."

When an AI contractor ingests this, it parses the tokens probabilistically. Because the language is highly specific to software engineering jargon ("churn," "scoped," "hot zone"), the AI is more likely to hallucinate or misinterpret the boundary conditions if it encounters an edge case. Simplifying the text reduces the token-space complexity, fundamentally aligning the AI's processing with the Feynman Technique's requirement to break down complex ideas into simple language, thereby activating more robust cognitive retrieval pathways.[16]

3. Synthesis: The Stela Dialectic (The Fifth-Grader Translation Matrix)

To satisfy the objective of synthesizing all aspects of human-readable text within Stela into a coherent dialectic understandable by a fifth grader, the following comprehensive translation matrices apply Feynman's methodology.[2] This synthesis strips away the "names of the bird" and focuses entirely on "what the bird is doing," reimagining the highly technical repository as a physical, tangible workshop.[7]

In a physical workshop, there are tools, workbenches, rules to keep people safe, logbooks to track what was built, and inspectors to ensure quality. By mapping digital concepts to these physical realities, knowledge ceases to be fragile.[12]

3.1 Translating the Filing Doctrine (The Workshop Layout)

The physical directory structure of the Stela repository (ops/, docs/, projects/, etc.) represents the physical layout of a carpentry workshop.[9]

Current Stela Jargon	**Feynman 5th-Grader Translation**	**Underlying Conceptual Analogy**
ops/ = Run (binaries, manifests, automation).	**The Tool Shed:** This is where we keep the hammers, saws, and robots that do the heavy lifting. You don't build things *in* the tool shed; you just take tools from here to use them.	Tools act upon material but are not the material itself. Executable isolation.
docs/ = Explain (manuals and rationale).	**The Instruction Manuals:** This is the bookshelf in the corner. It tells you how to safely use the tools and what the rules of the shop are.	Schematics, wayfinding, and guidelines governing behavior.
projects/ = Work (payload code).	**The Workbench:** This is where you actually build the toy car or fix the radio.[22] All	The physical space where raw materials are

	your active building, cutting, and gluing happens right here.	manipulated; the payload.
var/tmp/ = Resume (ephemeral worker scratch).	**The Scratchpad:** A piece of scrap paper you use to do math. When you are done figuring out the answer, you throw the paper away.	Temporary memory allocation; ephemeral, disposable execution state.
storage/ = Payload (handoff, dumps, dp).	**The Filing Cabinet:** Where we keep photographs of the workbench and the receipts that prove we bought the materials. We do not edit things in the cabinet; we just store them.	Immutable state capture, handoff artifacts, and transaction history.
archives/ = Cold (long-lived draft).	**The Basement:** Old boxes of things we finished a long time ago. We keep them just in case we need to remember how we built something last year.	Long-term, read-only data retention and deep ledger history.
opt/ = Isolate (JIT-only content).	**The Secret Blueprints:** Special instructions we only pull out when we are doing a very specific, dangerous job. Otherwise, they stay locked away so they don't clutter the workbench.	Context-window hygiene; preventing persistent state contamination.

3.2 Translating the Axioms and Canon (PoT.md)

The PoT.md is the Constitution. It is the absolute law of the workshop.[9] It does not ask nicely; it simply states how the universe works.

Current Stela Jargon	**Feynman 5th-Grader Translation**	**Underlying Conceptual Analogy**

SSOT (Single Source of Truth)	**One True Map:** If we want to know where the treasure is, there is only *one* real map. If someone draws a copy of the map on a napkin, we don't trust the napkin. We always go back to the real map.	Data normalization; avoiding duplicated, conflicting data arrays that cause logic loops.
Drift (Divergence between canon and state)	**Lying to Yourself:** This is when the map says the chair is in the corner, but the chair is actually in the middle of the room. When the map and the room don't match, we stop everything and fix the map.	State synchronization; maintaining absolute scientific integrity and intellectual honesty.[18]
Precedence (PoT is final authority)	**The Boss's Rule:** If someone tells you to do something, but the big rulebook on the wall says you shouldn't, you point at the rulebook and say, "No." The wall rulebook always wins.	Hierarchical conflict resolution, strict override protocols, and boundary enforcement.
Routing Closeout	**Cleaning Up:** You are not done building the toy until you sweep the floor, put the tools back in the shed, and write down what you did. Leaving a mess is a failure.	Lifecycle management, garbage collection, and state equilibrium.

3.3 Translating the Ledgers (SoP.md, PoW.md, TASK.md)

The ledgers are the memory of the system. Without them, the system has amnesia, resetting to zero every time an AI agent's context window clears.[9]

Current Stela Jargon	Feynman 5th-Grader	Underlying Conceptual

	Translation	Analogy
TASK.md (Active work surface)	**The Chore List:** A sticky note on the fridge that tells you exactly what chores you need to do today. You cross them off as you go. You don't add new chores to it unless the boss says so.	Ephemeral execution state; the active processing thread and dispatch contract.
SoP.md (State of Play / History)	**The Workshop Diary:** A storybook where we write down *why* we decided to build a birdhouse instead of a doghouse. It tells the story of our choices so people in the future know what we were thinking.	Narrative memory; contextual rationale for past decisions and system evolution.
PoW.md (Proof Ledger)	**The Box of Receipts:** When you go to the store to buy wood, you bring back a receipt. This file doesn't tell a story; it just points to the exact drawer where the receipt is kept to prove you did the work.	Cryptographic or evidentiary pointers; unarguable proof of action required for auditing.

3.4 Translating the Tooling, Gates, and Enforcement (The Robots)

Stela uses automated bash scripts (ops/bin/open, ops/bin/certify, tools/lint/*) to enforce the rules without human intervention.[9] Feynman would explain these algorithms not as "deterministic execution validations," but as specialized robots or helpers acting within the workshop.

Current Stela Jargon	Feynman 5th-Grader Translation	Underlying Conceptual Analogy
ops/bin/open (Freshness	**Taking a Polaroid:** Before	Snapshotting environment

state)	you start working, you take a quick photograph of the room. This proves exactly what the room looked like before you touched anything, so nobody can blame you for messes that were already there.	variables to prevent state contamination and establish a baseline.
ops/bin/dump (State serialization)	**Packing the Backpack:** Gathering up all the tools, maps, and instructions you will need for the day and putting them into one bag so you don't have to keep running back to the house while you work.	Assembling localized, bundled context for disconnected processing and AI context ingestion.
ops/bin/certify (Generated receipts)	**The Inspector:** A robot that walks up to your finished birdhouse, shakes it to make sure it doesn't fall apart, and prints out a golden ticket that says, "This was built correctly."	Automated testing, validation, and cryptographic signing of completion states.
tools/lint/integrity.sh	**The Bouncer:** A guard standing at the door. If you try to bring a tool into the shop that isn't on his approved list, he blocks the door and yells at you to stop.	Bounded execution environments, directory isolation, and strict allowlist enforcement.
tools/lint/style.sh	**The Spelling Teacher:** A teacher who checks your homework to make sure you didn't use big, confusing words just to sound smart. If you use jargon, she makes you write	Enforcing cognitive simplicity, linguistic determinism, and the Feynman technique.

	it again simply.	

3.5 Translating Staffing and Roles

Stela defines strict boundaries between the human and the machine.[9] Explaining this to a child requires establishing clear roles of authority.

Current Stela Jargon	Feynman 5th-Grader Translation	Underlying Conceptual Analogy
Operator (Human)	**The Owner:** The person who owns the workshop. Only the Owner is allowed to turn the power on, sign the checks, or officially accept a finished project.	Root-level administrative privileges; the final arbiter of state mutations (merging PRs).
Integrator (Lead AI)	**The Foreman:** The boss who walks around the shop making sure everyone is following the rules and reading the blueprints correctly. The Foreman writes the Chore List but doesn't swing the hammers.	Orchestration, structural integrity auditing, and prompt generation.
Contractor (Guest AI)	**The Builder:** The worker who actually picks up the hammer and builds the birdhouse based strictly on the Foreman's Chore List. The Builder never makes up their own chores.	Execution of logic tasks, drafting implementations, and scoped code generation.

4. Strategic Options for Implementation

Transforming the textual corpus of an entire repository to align with Feynman's philosophy of explainability is a massive structural undertaking. The primary risk lies in the tension between

machine-readability and human-readability. If a document is over-simplified, critical machine-readable contracts (which automated parsers and linters rely upon to function) may fail. If it is under-simplified, it retains the "pompous fool" jargon that Feynman despised, maintaining the cognitive fragility the system seeks to eliminate.[18]

The following strategic paths present distinct mechanisms for achieving the objective of total textual simplification across the Stela environment.

Option A: The Parallel "Rosetta Stone" Explanatory Layer

This approach leaves all existing technical specifications, the PoT.md, and ops/ scripts completely intact and untouched. It addresses the explainability mandate by creating an entirely new, parallel directory structure (e.g., docs/explain/ or a specialized wiki) containing 1:1 translations of every complex concept into the 5th-grader analogies detailed in Section 3.

- **Pros:**
  - **Zero Regression Risk:** Because the canonical files are untouched, there is absolutely zero risk of breaking existing CI/CD gates, parsers, or automated tools/lint/* scripts that rely on exact string matches.
  - **Preserves Legalistic Tone:** Preserves the strict, legally binding nature of the current PoT.md for edge-case dispute resolution between highly technical operators.
- **Cons:**
  - **Violates Core Axioms:** Directly violates the PoT.md Section 1.2 SSOT axiom ("one canonical file per domain; other mentions are pointers").[9] Creating parallel documentation introduces massive "Drift" risk. If the code changes, the explanation layer will inevitably fall out of sync, creating the exact incoherence the system seeks to avoid.
  - **Fails the Feynman Test:** Philosophically, this approach fails the fundamental Feynman test. If the primary text cannot be explained simply, it is the primary text that is flawed, not the reader.[5] Appending an explanation to bad writing does not fix the bad writing; it merely adds bulk.
- **Risk Level:** Low operational risk (no broken pipelines), High architectural risk (guaranteed long-term documentation drift).

Option B: Foundational Narrative Refactoring (The Feynman Override)

This approach involves a destructive, in-place rewrite of all human-facing markdown files across the repository (PoT.md, docs/MANUAL.md, docs/MAP.md, ledger entry instructions, and docs/ops/specs/*). The dense academic jargon is entirely stripped and permanently replaced with the literal 5th-grader analogies, treating the entire repository as a physical workshop.

- **Pros:**
  - **Aligns with Mandates:** Perfectly aligns with PoT.md Section 4.2: "Relatability Mandate: Universal accessibility via vernacular optimization and anecdotal grounding".[9]
  - **Reduces Cognitive Load:** Drastically reduces cognitive load for human operators and

token-ambiguity for AI LLM contexts, leading to fewer hallucinations and higher execution fidelity.[16] When an LLM reads "The Builder never makes up their own chores," it understands boundary limits much better than "Contractors do not create or switch branches."

- **Cons:**
  - **Extensive Tooling Updates:** Requires extensive, synchronized modifications to the tools/lint/truth.sh and tools/lint/style.sh scripts, which currently search for exact, dense phrasing to validate documentation compliance.
  - **Cultural Friction:** May initially alarm engineers and operators accustomed to traditional, dense, passive-voice software engineering documentation, requiring a brief period of cultural adaptation.
- **Risk Level:** Moderate operational risk (requires careful synchronization with linters during the PR phase), Low architectural risk (eliminates drift by maintaining a single source of truth).

Option C: Tool-Assisted "Just-In-Time" (JIT) Translation

Instead of rewriting the core text at rest, this approach leaves the dense files in place but updates the ops/bin/open and ops/bin/dump binaries.[9] When an operator or AI begins a session, the tools utilize a lightweight, localized parsing step (or predefined substitution regex) to automatically translate dense CONTEXT.md materials into the 5th-grader dialectic purely for the temporary var/tmp/ session files and the OPEN prompt.

- **Pros:**
  - **Compact Source:** The source code and canonical documentation remain highly technical, compact, and optimized for minimal byte size.
  - **Tailored Ingestion:** The execution environment receives perfectly tailored, simple instructions without permanently altering the repository state.
- **Cons:**
  - **Latency and Overhead:** Adds computational overhead and potential latency to the open and dump commands, slowing down the initialization phase of every single task.
  - **Philosophical Violation:** Violates the "Absolute literalism" mandate.[9] By having the machine store one reality on disk but present a different reality to the worker, the system introduces a layer of abstraction that Feynman vehemently opposed. "What you see is what you get" is lost.
- **Risk Level:** High operational risk (fragility in automated translation pipelines; risk of translation errors altering strict operational boundaries).

Recommendation

Option B (Foundational Narrative Refactoring) is the singular path that honors both Feynman's intellectual integrity regarding clear communication [20] and Stela's fundamental Single Source of Truth (SSOT) axiom.[9] The complexity of Stela's current text is a bug, not a feature. As Feynman noted, "Anyone can make a subject complicated but only someone who

understands can make it simple".[5] The text itself must be permanently cured of its jargon to ensure long-term system stability and true explainability.

5. Formulated Plan: The Feynman Refactor PR Sequence

To address ALL text contained within Stela without destabilizing the repository or breaking the automated gates, the refactoring must occur across a meticulously sequenced series of Pull Requests (PRs). Each PR will adhere strictly to the "Immutable DP Workflow" (ops/bin/open -> ops/bin/draft -> ops/bin/certify -> PR) to ensure cryptographic proof of the changes.[9]

The plan spans six deliberate phases, ensuring that dependencies (like linters) are updated in tandem with the text they monitor.

Phase 1: The Constitution and Governance Alignment (PR-01)

- **Target Files:** PoT.md, docs/GOVERNANCE.md, README.md.
- **Objective:** Rewrite the core laws of the system using tangible analogies, eliminating pseudo-academic jargon while preserving the strict behavioral logic and physical constraints of the repository.
- **Execution Logic:**
  1. Replace instances of abstract philosophy (e.g., "Truth: primitive predicate governed by specific rules") with explicit, action-based physical analogies ("The Rules of the Workshop: We only trust what we can see and prove, not what we assume.").[9]
  2. Rewrite Section 4.2 "Behavioral Logic Standard." Specifically, translate the dense "Relatability Mandate" into: "Speak simply. Use stories of real-world objects to explain abstract code. If a 10-year-old cannot understand the instruction, rewrite it."
  3. Update the "System Failure States" table in PoT.md Section 2. Instead of "Dirty State / Uncommitted changes," use "Leaving tools on the workbench. Stop work. Clean up before starting a new job."
- **Validation Gates:** Run bash tools/lint/style.sh and bash tools/lint/truth.sh to ensure the new simplistic vocabulary does not trigger legacy word blacklists. Execute ./ops/bin/certify to generate the new RESULTS receipt.

Phase 2: Wayfinding and Command Mechanics (PR-02)

- **Target Files:** docs/MANUAL.md, docs/MAP.md, docs/INDEX.md.
- **Objective:** Simplify the operator interfaces and continuity mapping so navigation is intuitive and does not require rote memorization of complex paths, directly applying the "Map of the Cat" philosophy.[14]
- **Execution Logic:**
  1. Refactor docs/MAP.md. Explicitly state that the map is a tool to find the territory, not the territory itself. Translate the Directory Index (e.g., ops/ = Run, docs/ = Explain) using

the Tool Shed, Instruction Manuals, and Basement analogies detailed in Section 3.1.
2. In docs/MANUAL.md, rewrite the "Execution Cycle" and "Closeout Cycle." Replace terms like "Generates canonical DP block" with "Prints a fresh Chore List."
3. Clarify the ops/bin/dump mechanics. Remove phrases like "Serializes the platform" and replace them with "Takes a snapshot of the entire workshop and puts it in a folder so you can look at it later".[9]

- **Validation Gates:** Execute ./ops/bin/map --check and ./ops/bin/help doctrine to ensure automated menu generation accurately parses the newly simplified headings without breaking the CLI tool.

Phase 3: The Ledger Contracts and Epistemology (PR-03)

- **Target Files:** docs/ops/specs/surfaces/pow.md, docs/ops/specs/surfaces/sop.md, docs/ops/specs/surfaces/task.md, ops/src/surfaces/task.md.tpl.
- **Objective:** Clarify the epistemological difference between narrative history (SoP), hard cryptographic proof (PoW), and active intent (TASK), ensuring no operator ever confuses them again.
- **Execution Logic:**
  1. Implement the Feynman differentiation between "knowing the name" and "knowing the thing".[21]
  2. Revise sop.md to explicitly state: "This is the Workshop Diary. It is the story of what we did. Write it so a 5th grader understands *why* we changed the code. Do not paste raw code here."
  3. Revise pow.md to state: "This is the Box of Receipts. Do not tell a story here. Only provide the exact file paths that prove the work was done."
  4. Update the canonical TASK.md template preamble. Change "Session State is pointer-first" to "Do not write the data here; only write the map to where the data lives."
- **Validation Gates:** Run bash tools/lint/task.sh and bash tools/lint/schema.sh to ensure container schema validation does not break due to preamble simplification.

Phase 4: Binary Specification Demystification (PR-04)

- **Target Files:** docs/ops/specs/binaries/*.md (e.g., certify.md, draft.md, dump.md, prune.md).
- **Objective:** Translate the deep technical specs of the repository's executables into simple, mechanical cause-and-effect documents, focusing on what the tool physically does to the repository.
- **Execution Logic:**
  1. Address the "First Principles Rationale" sections. Currently, they use phrases like "protects PoT truth and anti-drift axioms by serializing selected tracked content".[9]
  2. Translate to Feynman-style logic: "Why do we have ops/bin/dump? Because if people keep their work hidden on their own computers, no one else can see the truth. This

tool gathers all the visible work into one public box so everyone looks at the exact same reality."
3. Simplify the "Mechanics and Sequencing" sections. Replace passive voice and complex clauses with chronological, active-voice steps (e.g., "First, the tool checks the doors. Second, it reads the list.").

- **Validation Gates:** Run bash tools/lint/schema.sh and bash tools/lint/style.sh across the specs directory to ensure the four canonical H2 headings required by the specs are maintained while the content beneath them is simplified.

Phase 5: Factory Definition and JIT Skill Refactoring (PR-05)

- **Target Files:** opt/_factory/agents/*.md, opt/_factory/skills/*.md, opt/_factory/tasks/*.md, docs/ops/specs/definitions/*.md.
- **Objective:** Rewrite the "Scope", "Pointers", and "Guardrails" of all JIT (Just-In-Time) knowledge files to ensure instructions given to AI agents are universally accessible, reducing token-space complexity and preventing hallucination.
- **Execution Logic:**
  1. Review files like opt/_factory/skills/s-learn-06.md (Hot Zone Forensics).[9] Translate instructions like "Measure churn with git log scoped to touched files" into "Look at the history of the file. If it has been broken and fixed many times recently, treat it like a hot stove. Be careful."
  2. Simplify the Agent profiles. Instead of "Validates factory candidate and promotion pointer workflows," use "Your job is to check the map. Make sure every arrow on the map points to a real file, not a dead end."
- **Validation Gates:** Run bash tools/lint/factory.sh, bash tools/lint/agent.sh, and bash tools/lint/skill.sh to guarantee that structural markers (which the linters depend on) are preserved even as the surrounding narrative text is simplified.

Phase 6: Hardening the Simplicity (The Linter Update) (PR-06)

- **Target Files:** tools/lint/style.sh, docs/ops/specs/tools/lint/style.md.
- **Objective:** Permanently encode Feynman's explainability philosophy into the repository's automated defense mechanisms, preventing future jargon from creeping back into the ecosystem.[12]
- **Execution Logic:**
  1. Expand the JARGON_BLACKLIST inside tools/lint/style.sh.[9] Add overly academic terms like "serialization," "vernacular optimization," "ephemeral," "paradigm," and "leverage" (when used as a verb).
  2. Update the error messages emitted by the script. Instead of outputting "Linguistic Precision violation detected," the script should output: "Jargon detected. Explain this simpler. Imagine you are teaching a 10-year-old."
  3. Integrate a lightweight syllable/word-length counter function in the bash script to act as a rudimentary readability proxy, failing CI/CD gates if sentences repeatedly exceed

maximum complexity thresholds.

- **Validation Gates:** Execute the updated tools/lint/style.sh against the entire newly refactored repository to prove a clean state. Run ./ops/bin/certify to seal the final artifact.

6. Conclusion

The Stela repository currently functions as an architectural marvel of deterministic control, yet it suffers from an acute case of linguistic "Cargo Culting".[23] It relies on the highly technical names of things rather than the physical mechanisms of things.[7] By systematically applying Richard Feynman's epistemological lens—stripping away abstraction, relying heavily on physical analogies, and designing text for a fifth-grade comprehension level—the system can eliminate the fragility of rote memorization that plagues both human operators and Large Language Models.[11]

The Formulated Plan, spanning six distinct and sequentially dependent Pull Requests, provides a safe, structurally sound pathway to execute Foundational Narrative Refactoring. By treating the repository as a physical workshop, the bash tools as physical implements, and the markdown ledgers as literal diaries and receipt boxes, Stela will successfully bridge the gap between machine-readable strictness and human-readable simplicity. The result will be a repository where truth is not just strictly governed, but intuitively and universally understood.

Works cited

1. Surely You're Joking, Mr. Feynman! - Wikipedia, accessed February 21, 2026, https://en.wikipedia.org/wiki/Surely_You%27re_Joking,_Mr._Feynman!
2. The Feynman Technique Simplified: How to Help Kids Learn Better - Education Briefs, accessed February 21, 2026, https://www.thethinkacademy.com/blog/edubriefs-feynman-technique-simplified-parents/
3. Richard Feynman: Knowing the Name of Something, accessed February 21, 2026, https://fs.blog/richard-feynman-knowing-something/
4. The Feynman Learning Technique - Farnam Street, accessed February 21, 2026, https://fs.blog/feynman-learning-technique/
5. Feynman Technique: The Ultimate Guide to Learning Anything Faster - Farnam Street, accessed February 21, 2026, https://fs.blog/feynman-technique/
6. Feynman's "Knowing" anecdote – I n q u i r y - UBC Blogs, accessed February 21, 2026, https://blogs.ubc.ca/edutara/feynmans-knowing-anecdote/
7. Classic Richard Feynman - The Difference Between Knowing the Name of Something and ... - Nasdaq, accessed February 21, 2026, https://www.nasdaq.com/articles/classic-richard-feynman-difference-between-knowing-name-something-and-knowing-something
8. accessed February 21, 2026, https://fs.blog/feynman-learning-technique/#:~:text=You%20only%20know%20something%20about,t%20mean%20you%20understand%20it.

9. dump-full-main-692b5321.txt
10. Richard Feynman On Education in Brazil | PDF | Polarization (Waves) | Socrates - Scribd, accessed February 21, 2026, https://www.scribd.com/document/514125538/Richard-Feynman-on-education-in-Brazil
11. The Day Feynman Realized Students Knew NOTHING - (Brazil Lecture, 1952) - YouTube, accessed February 21, 2026, https://www.youtube.com/watch?v=T1PuGEsE4po
12. "Surely You're Joking, Mr. Feynman!" Quotes - Goodreads, accessed February 21, 2026, https://www.goodreads.com/work/quotes/321174-surely-you-re-joking-mr-feynman-adventures-of-a-curious-character
13. Richard Feynman On The Education System in Brazil - Musings of an Enlightened Idiot, accessed February 21, 2026, https://enlightenedidiot.net/random/feynman-on-brazilian-education-system/
14. Mathematics_ Feynman, Richard - Surely You'... - FSU | Department of Chemistry & Biochemistry, accessed February 21, 2026, https://www.chem.fsu.edu/chemlab/isc3523c/feyn_surely.pdf
15. Feynman's Hack and the Map of the Cat - Safal Niveshak, accessed February 21, 2026, https://www.safalniveshak.com/feynman-map-of-the-cat/
16. Feynman Technique for Kids: Brain-Based Learning Guide, accessed February 21, 2026, https://www.kutubooku.com/post/the-feynman-technique-for-kids-learning-reading-understanding
17. The Feynman Technique: How to Learn Anything Quickly - Todoist, accessed February 21, 2026, https://www.todoist.com/inspiration/feynman-technique
18. "Surely You're Joking, Mr. Feynman!" Quotes - Goodreads, accessed February 21, 2026, https://www.goodreads.com/work/quotes/321174-surely-you-re-joking-mr-feynman-adventures-of-a-curious-character?page=6
19. Excerpts From Surely You're Joking Mr. Feynman And My Thoughts About Them, accessed February 21, 2026, https://enlightenedidiot.net/musings/excerpts-surely-you-are-joking-mr-feynman/
20. SURELY YOU'RE JOKING MR. FEYNMAN! | by A dummy's guide - Medium, accessed February 21, 2026, https://medium.com/@gursahay24/surely-youre-joking-mr-feynman-280b8622acaf
21. “I learned very early the difference between knowing the name of something and knowing something.” — Richard Feynman [1024×1024] : r/QuotesPorn - Reddit, accessed February 21, 2026, https://www.reddit.com/r/QuotesPorn/comments/sne0wz/i_learned_very_early_the_difference_between/
22. Surely You're Joking Mr. Feynman by Richard Feynman - dltn.io, accessed February 21, 2026, https://www.dltn.io/posts/surely-youre-joking-mr-feynman

23. Richard Feynman's advice for science in 1974 resonates still - World Bank Blogs, accessed February 21, 2026, https://blogs.worldbank.org/en/impactevaluations/richard-feynmans-advice-science-1974-resonates-still